

RAG Architecture Blocker Report

1. Objective

We are trying to clarify the RAG architecture for the next stage of the project. The final product is expected to support a Felt dashboard where users can click on a parcel/building space and understand whether a laneway home can likely be built.

Our current understanding is that the GIS side should calculate the buildable envelope using zoning rules such as front setback, rear setback, side setbacks, lot coverage, and available height/FAR constraints. RAG should mainly support rule lookup, source matching, and explanation.

2. Attempts

We reviewed the latest project direction and compared it with our current rule extraction work. We also started thinking about how RAG would connect to the GIS workflow and Felt dashboard.

So far, we considered this basic workflow:

Bylaw documents → RAG retrieval/source matching → structured rule table → GIS buildable envelope calculation → Felt dashboard explanation

We also did a quick planning-level review of possible RAG model options, including OpenAI, Claude, Gemini, Cohere, and local/open-source models.

3. Evidence

From the meeting and latest feedback, it seems the immediate deliverable should focus on parcel-level buildable area/envelope, not exact building-plan testing.

We also observed that RAG can become unclear if we do not define its role early. For example, RAG could be used for rule extraction and explanation, but it should probably not be responsible for geometry calculations or final legal/buildability decisions.

Another observation is that source matching is critical. If the system says a setback or lot coverage rule applies, we need to connect that rule back to the bylaw section/code, page if available, and supporting text snippet.

4. Diagnosis

The blocker is not that we cannot build a RAG prototype. The blocker is that we need to confirm what the RAG prototype is supposed to do before implementation.

If we make RAG responsible for too much, the system may become unreliable or hard to explain. A safer design is likely:

- GIS/rule logic calculates the buildable envelope.
- The rule table stores verified zoning constraints.
- RAG helps retrieve, source-match, and explain the rules.
- Felt visualizes the parcel-level result.

We also need clarification on model/tooling cost. If paid model calls, embeddings, vector storage, or extra computing are required, we should know whether those costs are covered and whether there is a preferred provider.

5. Proposed Next Steps

1. Build a small RAG prototype for one municipality and one rule group first, such as setbacks and lot coverage for laneway homes.
2. Update the rule table so each rule has machine-readable fields such as rule key, operator, value, unit, condition, source document, bylaw code/section, supporting snippet, and review status.
3. Keep the GIS workflow separate from RAG by using verified rules to calculate buildable envelopes, while RAG supports explanation and source lookup.
4. Ask for recommended learning resources because RAG source matching, GIS envelope calculation, and Felt dashboard integration are not fully covered in our class.

6. Decision Ask

1. Can you confirm that RAG should mainly support rule lookup, source matching, and explanation, while GIS/rule logic handles the actual buildable-envelope calculation?
2. If we need paid model usage, embeddings, vector storage, or computing resources for the RAG prototype, would those costs be covered? Also, do you have a preferred model/provider such as OpenAI, Claude, Gemini, Cohere, or local models?
3. Could you recommend 2–3 high-value tutorials, books, videos, documentation pages, or example projects to help us understand RAG source matching, structured rule extraction, GIS buildable-envelope calculation, and Felt dashboard workflow?